

Configuración del entorno de desarrollo

Flujo de trabajo con Git y GitHub

Desarrollo Web Backend · 1 sesión · Gestión 02/2026 · M. Sc. Huáscar F. Gonzales G.

Contenido

- | | | | |
|----|------------------------------------|----|-------------------------------|
| 01 | Identificación de la práctica | 11 | Instalación de Postman |
| 02 | Objetivos | 12 | Fundamentos de Git |
| 03 | Introducción teórica | 13 | GitHub y autenticación SSH |
| 04 | Requisitos previos y convenciones | 14 | Estructura del repositorio |
| 05 | Instalación y configuración de Git | 15 | Ejercicio práctico integrador |
| 06 | Instalación de PHP 8.4 | 16 | Checklist de verificación |
| 07 | Instalación de Composer | 17 | Solución de problemas comunes |
| 08 | Instalación de PostgreSQL 18 | 18 | Entregable y evaluación |
| 09 | Instalación de Node.js 24 LTS | 19 | Bibliografía y recursos |
| 10 | Visual Studio Code y extensiones | | |

01 Identificación de la práctica

Campo	Detalle
Facultad	Ciencias Puras
Carrera	Ingeniería Informática
Asignatura	Desarrollo Web Backend
Sigla y código	INF 560
Unidad	Unidad 1 — Fundamentos del Desarrollo Web
Nº de guía	Guía de Laboratorio Nº 01
Tema	Configuración del entorno de desarrollo y flujo de trabajo con Git y GitHub
Duración estimada	1 sesión
Modalidad	Práctica individual en laboratorio / trabajo independiente
Docente	M. Sc. Huáscar Fedor Gonzales Guzmán

02 Objetivos

Objetivo general

Configurar un entorno de desarrollo backend completo y funcional, e integrar un flujo de trabajo profesional basado en control de versiones con Git y en la plataforma colaborativa GitHub, que servirá de base para todas las prácticas de la asignatura durante la gestión 02/2026.

Objetivos específicos

- Instalar y verificar las herramientas base del desarrollo backend: Git, PHP, Composer, PostgreSQL, Node.js, Visual Studio Code y Postman.
- Comprender el rol de cada herramienta dentro de la arquitectura de una aplicación web moderna.
- Configurar Git con la identidad del estudiante y aplicar el ciclo básico de control de versiones (working directory, staging area y repositorio).
- Generar llaves SSH y autenticar el equipo local contra GitHub de forma segura.
- Crear el repositorio personal de la asignatura con una estructura de carpetas y archivos base estandarizada.

- Realizar el primer ciclo de confirmación y publicación de cambios (commit + push) hacia un repositorio remoto.

03 Introducción teórica

El entorno de desarrollo backend

El desarrollo del lado del servidor (backend) requiere un conjunto de herramientas que trabajan de forma coordinada: un intérprete del lenguaje, un gestor de dependencias, un motor de base de datos, un editor de código y utilidades para probar y versionar el trabajo. A diferencia del frontend, donde basta un navegador, el backend ejecuta lógica de negocio, accede a datos y expone servicios; por ello el primer paso y el más frecuentemente subestimado es construir un entorno reproducible en el que todos esos componentes se comuniquen correctamente.

En esta guía instalaremos el stack que acompañará a la asignatura. Aunque el framework principal será Laravel (PHP), el plan de la gestión 02/2026 incorpora también Node.js/Express, por lo que preparamos ambos ecosistemas desde el inicio.

Arquitectura cliente-servidor

Toda aplicación web opera bajo el modelo cliente-servidor: el cliente (típicamente el navegador) envía una petición HTTP y el servidor la procesa y devuelve una respuesta. El backend es el software que corre en el servidor. En nuestro entorno de desarrollo, la propia máquina del estudiante actúa simultáneamente como cliente y servidor (`localhost`), lo que permite desarrollar y probar sin necesidad de un servidor remoto.

Definición · localhost y puertos

`localhost` (dirección 127.0.0.1) es el nombre que la máquina usa para referirse a sí misma. Un servidor de desarrollo se expone en un puerto, por ejemplo `http://localhost:8000` para Laravel o `http://localhost:3000` para Express. El puerto identifica qué proceso atiende la petición dentro del mismo equipo.

Rol de cada herramienta

La siguiente tabla resume qué aporta cada componente que instalaremos y por qué es necesario:

Herramienta	Rol dentro del entorno
Git	Sistema de control de versiones. Registra el historial de cambios del código de forma local.
GitHub	Plataforma remota que aloja los repositorios Git y habilita la colaboración y la entrega de trabajos.
PHP	Intérprete del lenguaje del lado del servidor; ejecuta la lógica de Laravel.
Composer	Gestor de dependencias de PHP; instala Laravel y sus paquetes (equivalente a npm en Node).
PostgreSQL	Sistema gestor de base de datos relacional donde se almacenan y consultan los datos.
Node.js y npm	Entorno de ejecución de JavaScript del lado del servidor; compila los recursos frontend (Vite) de Laravel.
Visual Studio Code	Editor de código con extensiones para PHP, Laravel, Git y bases de datos.
Postman	Cliente para construir y probar peticiones HTTP y API REST sin necesidad de un navegador.

04 Requisitos previos y convenciones

Requisitos mínimos

- Equipo con al menos 8 GB de RAM (recomendado) y 15 GB de espacio libre en disco.
- Sistema operativo: Windows 10/11 (64 bits), Linux (Ubuntu 22.04+ o derivada) o macOS 13+.
- Permisos de administrador para instalar software.
- Conexión estable a Internet.
- Una cuenta de correo electrónico vigente (se usará para GitHub y para la configuración de Git).

Convenciones de esta guía

A lo largo del documento se emplean las siguientes convenciones para los comandos de terminal:

- Los bloques oscuros contienen comandos que deben escribirse en la terminal. Escriba una línea a la vez y presione Enter.
- El símbolo `$` o `>` al inicio representa el prompt; no debe copiarse.
- El texto entre `_backticks` dentro de un párrafo indica un comando, archivo o valor literal.
- Las instrucciones específicas de cada sistema operativo aparecen bajo las etiquetas Windows, Linux y macOS.

Nota · terminal según el sistema

En Windows usaremos PowerShell o, preferentemente, Git Bash (se instala junto con Git). En Linux y macOS usaremos la terminal del sistema. Tras instalar una herramienta que modifique el PATH, cierre y vuelva a abrir la terminal para que los cambios tengan efecto.

05 Instalación y configuración de Git

Definición · control de versiones

Un sistema de control de versiones (VCS) registra los cambios de un conjunto de archivos a lo largo del tiempo, permitiendo recuperar versiones anteriores, comparar cambios y trabajar en paralelo. Git es un VCS distribuido: cada copia del repositorio contiene el historial completo.

Instalación

Windows

- Descargue el instalador desde <https://git-scm.com/download/win>.
- Ejecute el instalador y acepte las opciones por defecto, salvo en “Choosing the default editor”, donde puede seleccionar Visual Studio Code si ya lo tiene instalado.
- En “Adjusting your PATH environment”, deje la opción recomendada (Git desde la línea de comandos y software de terceros).

Linux (Ubuntu / Debian)

```
bashCopiar  
  
sudo apt update  
sudo apt install -y git
```

macOS

Con Homebrew instalado (<https://brew.sh>):

```
bashCopiar  
  
brew install git
```

Verificación

```
terminalCopiar  
  
$ git --version
```

Salida esperada (la versión menor puede variar):

```
salidaCopiar  
  
git version 2.46.0
```

Configuración inicial

Git necesita saber quién realiza cada cambio. Configure su identidad de forma global (una sola vez por equipo). Use su nombre real y el correo con el que creará su cuenta de GitHub:

```
terminal — git configCopiar  
  
git config --global user.name "Nombre Apellido"  
git config --global user.email "correo@example.com"  
git config --global init.defaultBranch main  
git config --global core.editor "code --wait"  
git config --global pull.rebase false
```

Verifique la configuración con:

```
terminal
```

[Copiar](#)

```
git config --global --list
```

Nota

El correo configurado aquí debe coincidir con el de su cuenta de GitHub para que sus commits se asocien a su perfil. La rama por defecto main es el estándar actual (antes se usaba master).

06 Instalación de PHP 8.4

Usaremos PHP 8.4, versión recomendada para producción a la fecha (agosto 2026). Laravel 13 requiere como mínimo PHP 8.3, por lo que 8.4 es una elección segura y estable. La última versión de la rama es 8.5, pero mantendremos 8.4 por compatibilidad amplia con paquetes.

Instalación

Windows (instalación manual)

- Ingrese a <https://windows.php.net/download> y descargue la versión “PHP 8.4 x64 Non Thread Safe” (archivo ZIP).
- Extraiga el contenido en `C:\php`.
- Copie el archivo `php.ini-development` y renombre la copia como `php.ini`.
- Abra `php.ini` y descomente (quite el `;` inicial) de `extension_dir = "ext"` y de las extensiones necesarias.

Extensiones a habilitar en `php.ini` (quitar el `;` al inicio de cada línea):

```
php.ini                                                                    Copiar

extension=curl
extension=fileinfo
extension=gd
extension=intl
extension=mbstring
extension=openssl
extension=pdo_pgsql
extension=pgsql
extension=exif
extension=zip
```

- Agregue `C:\php` al PATH: Panel de control → Sistema → Configuración avanzada → Variables de entorno → Path → Editar → Nuevo → `C:\php`.

Linux (Ubuntu / Debian)

```
bash — Ubuntu/Debian                                                    Copiar

sudo apt update
sudo apt install -y software-properties-common
sudo add-apt-repository -y ppa:ondrej/php
sudo apt update
sudo apt install -y php8.4 php8.4-cli php8.4-common php8.4-pgsql \
  php8.4-mbstring php8.4-xml php8.4-curl php8.4-zip php8.4-bcmath \
  php8.4-gd php8.4-intl
```

macOS

```
bash                                                                      Copiar

brew install php@8.4
brew link php@8.4 --force
```

Verificación

```
terminal                                                                  Copiar

$ php -v
```

Salida esperada:

```
salida Copiar

PHP 8.4.11 (cli) (built: ...) ( NTS )
Copyright (c) The PHP Group
Zend Engine v4.4.11, Copyright (c) Zend Technologies
```

Verifique que las extensiones para PostgreSQL estén cargadas:

```
terminal Copiar

$ php -m | grep pgsql
pdo_pgsql
pgsql
```

Si pdo_pgsql no aparece, revise que la extensión esté descomentada en php.ini (Windows) o instalada (Linux/macOS).

07 Instalación de Composer

Definición · gestor de dependencias

Composer administra las bibliotecas (paquetes) de las que depende un proyecto PHP. Descarga las versiones correctas, resuelve conflictos y las coloca en la carpeta vendor/. Es la herramienta con la que instalaremos Laravel.

Windows

- Descargue Composer-Setup.exe desde <https://getcomposer.org/download/>.
- Ejecute el instalador; detectará la ruta de php.exe (C:\php\php.exe). Acepte y finalice.

Linux / macOS

```
bash Copiar

php -r "copy('https://getcomposer.org/installer', 'composer-setup.php');"
php composer-setup.php
sudo mv composer.phar /usr/local/bin/composer
php -r "unlink('composer-setup.php');"
```

Verificación

```
terminal Copiar

$ composer -V
```

Salida esperada:

```
salida Copiar

Composer version 2.8.9 2026-...
```

08 Instalación de PostgreSQL 18

PostgreSQL es el sistema gestor de base de datos relacional que usaremos durante toda la asignatura. Instalaremos la versión 18 (última estable) y crearemos el rol y la base de datos que servirán al proyecto del semestre.

Instalación

Windows

- Descargue el instalador de EDB desde <https://www.postgresql.org/download/windows/>.
- Durante la instalación defina y anote la contraseña del superusuario postgres. Mantenga el puerto 5432.
- El instalador incluye pgAdmin 4 (cliente gráfico). Acepte instalarlo.
- Agregue `C:\Program Files\PostgreSQL\18\bin` al PATH para usar `psql` desde la terminal.

Linux (Ubuntu / Debian) — repositorio oficial PGDG

bash — Ubuntu/Debian

Copiar

```
sudo apt install -y curl ca-certificates
sudo install -d /usr/share/postgresql-common/pgdg
sudo curl -o /usr/share/postgresql-common/pgdg/apt.postgresql.org.asc \
  --fail https://www.postgresql.org/media/keys/ACCC4CF8.asc
echo "deb [signed-by=/usr/share/postgresql-common/pgdg/apt.postgresql.org.asc] \
  https://apt.postgresql.org/pub/repos/apt $(lsb_release -cs)-pgdg main" \
  | sudo tee /etc/apt/sources.list.d/pgdg.list
sudo apt update
sudo apt install -y postgresql-18
```

macOS

bash

Copiar

```
brew install postgresql@18
brew services start postgresql@18
```

Verificación

terminal

Copiar

```
$ psql --version
```

Salida esperada:

salida

Copiar

```
psql (PostgreSQL) 18.4
```

09 Instalación de Node.js 24 LTS

Instalaremos Node.js 24 (versión LTS activa, recomendada para proyectos nuevos). Node ejecuta JavaScript del lado del servidor y, en Laravel, compila los recursos frontend mediante Vite. Usaremos nvm (Node Version Manager) para poder cambiar de versión fácilmente.

Linux / macOS

bash

Copiar

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh | bash
# Cierre y reabra la terminal, luego:
nvm install 24
nvm use 24
nvm alias default 24
```

Windows — nvm-windows

- Descargue `nvm-setup.exe` desde <https://github.com/coreybutler/nvm-windows/releases>.
- Instálelo, abra una nueva terminal como administrador y ejecute los comandos:

```
terminal Copiar

nvm install 24
nvm use 24
```

Verificación

```
terminal Copiar

$ node -v
$ npm -v
```

Salida esperada (aproximada):

```
salida Copiar

v24.5.0
11.3.0
```

10 Visual Studio Code y extensiones

Instalación

- Descargue VS Code desde <https://code.visualstudio.com> e instálelo con las opciones por defecto.
- En Windows, marque “Add to PATH” y “Register Code as an editor for supported file types”.

Extensiones recomendadas

Con VS Code instalado y `code` disponible en el PATH, instale las extensiones ejecutando en la terminal:

terminal

Copiar

```
code --install-extension bmewburn.vscode-intelephense-client
code --install-extension laravel.vscode-laravel
code --install-extension onecentlin.laravel-blade
code --install-extension mikestead.dotenv
code --install-extension editorconfig.editorconfig
code --install-extension dbaeumer.vscode-eslint
code --install-extension esbenp.prettier-vscode
code --install-extension eamodio.gitlens
code --install-extension ms-ossdata.vscode-pgsql
```

Extensión	Utilidad
PHP Intelephense	Autocompletado, análisis y navegación de código PHP.
Laravel	Soporte específico para el framework: rutas, vistas y helpers.
Laravel Blade	Resaltado y snippets para las plantillas Blade.
DotENV	Resaltado de sintaxis del archivo <code>.env</code> .
EditorConfig	Aplica reglas de formato compartidas por el equipo.
ESLint / Prettier	Análisis y formateo automático de JavaScript.
GitLens	Historial y autoría de cada línea dentro del editor.
PostgreSQL	Cliente para conectarse y consultar la base de datos desde VS Code.

11 Instalación de Postman

Definición · Postman

Postman es un cliente que permite construir, enviar e inspeccionar peticiones HTTP (GET, POST, PUT, DELETE, etc.) sin escribir código. Es esencial para probar las API REST que construiremos en la Unidad 4.

Windows

- Descargue el instalador desde <https://www.postman.com/downloads/> o ejecute `winget install Postman.Postman`.

Linux

bash

Copiar

```
sudo snap install postman
```

macOS

bash

Copiar

```
brew install --cask postman
```

Abra Postman y cree una cuenta gratuita (opcional, útil para sincronizar colecciones). Como alternativa ligera dentro del editor, puede usar la extensión Thunder Client en VS Code.

12 Fundamentos de Git

Las tres áreas de Git

Git organiza el trabajo en tres áreas. Comprenderlas es clave para usar el control de versiones correctamente:

Área	Descripción
Working directory	Los archivos del proyecto tal como están en el disco, donde usted edita.
Staging area (index)	Zona intermedia donde se preparan los cambios que formarán parte del próximo commit.
Repositorio (.git)	Base de datos local donde Git guarda de forma permanente el historial de commits.

El flujo básico consiste en editar archivos (working directory), seleccionarlos con `git add` (staging) y confirmarlos con `git commit` (repositorio).

Ciclo básico de trabajo

terminal — git

Copiar

```
git init          # inicializa un repositorio en la carpeta actual
git status        # muestra el estado de los archivos
git add archivo.txt # agrega un archivo al staging
git add .         # agrega todos los cambios
git commit -m "mensaje" # confirma los cambios preparados
git log --oneline  # muestra el historial de commits
```

Ejemplo · anatomía de un buen mensaje de commit

Un mensaje debe describir QUÉ cambió y ser conciso. Correctos: Agrega estructura inicial del repositorio, Configura .gitignore para Laravel y Node. Evite mensajes vagos como “cambios” o “update”.

El archivo .gitignore

No todo debe versionarse. Las dependencias (vendor/, node_modules/) se reinstalan con Composer/npm, y las credenciales (.env) no deben publicarse. El archivo .gitignore le indica a Git qué ignorar. Crearemos uno preparado para proyectos Laravel y Node:

.gitignore

Copiar

```
# Dependencias
/vendor/
/node_modules/

# Entorno y credenciales
.env
.env.backup
.phpunit.result.cache

# Laravel
/public/build/
/public/storage
/storage/*.key
/bootstrap/cache/*.php

# Sistema operativo / editor
```

```
.DS_Store  
Thumbs.db  
*.log  
/.idea
```

13 GitHub y autenticación SSH

Creación de la cuenta

- Ingrese a <https://github.com> y regístrese con el mismo correo configurado en Git.
- Elija un nombre de usuario profesional (nombreapellido o similar); será parte de la URL de sus repositorios.
- Verifique su correo electrónico para activar la cuenta.

Generación de llaves SSH

Definición · autenticación SSH

SSH usa un par de llaves: una privada (que permanece en su equipo y nunca se comparte) y una pública (que se registra en GitHub). Al conectarse, GitHub verifica que ambas se corresponden, evitando escribir usuario y contraseña en cada operación.

Genere el par de llaves con su correo. Cuando se le pregunte por la ubicación y la frase de contraseña, puede presionar Enter para aceptar los valores por defecto:

terminal

Copiar

```
ssh-keygen -t ed25519 -C "correo@example.com"
```

Inicie el agente SSH y agregue la llave privada:

Linux / macOS / Git Bash

terminal

Copiar

```
eval "$(ssh-agent -s)"  
ssh-add ~/.ssh/id_ed25519
```

Muestre la llave pública para copiarla:

terminal

Copiar

```
cat ~/.ssh/id_ed25519.pub
```

Copie toda la línea (comienza con `ssh-ed25519`). En GitHub: Settings → SSH and GPG keys → New SSH key, pegue la llave, asígnele un título (por ejemplo “Laptop UATF”) y guarde.

Verificación de la conexión

terminal

Copiar

```
$ ssh -T git@github.com
```

Escriba yes si se le pregunta por la autenticidad del host. Salida esperada:

salida

Copiar

```
Hi usuario! You've successfully authenticated, but GitHub does not  
provide shell access.
```

Creación del repositorio remoto

- En GitHub, haga clic en New repository.
- Nombre: INF560-2026-02-Apellido (reemplace Apellido por el suyo).
- Visibilidad: Public (para que el docente pueda revisarlo).

- NO marque agregar README, .gitignore ni licencia (los crearemos localmente).
- Cree el repositorio y copie la URL SSH (`git@github.com:usuario/INF560-2026-02-Apellido.git`).

14 Estructura del repositorio del semestre

Estandarizaremos la estructura de carpetas para que todas las prácticas y el proyecto acumulativo estén organizados de forma coherente:

estructura

Copiar

```
INF560-2026-02-Apellido/
├── README.md
├── .gitignore
├── .editorconfig
├── laboratorios/
│   ├── gl01-entorno/
│   │   └── evidencias/
│   ├── gl02-github/
│   └── ...
├── proyecto/      # eventos-app (acumulativo desde la GL6)
└── recursos/      # apuntes, capturas y material de apoyo
```

Contenido sugerido para `.editorconfig` (unifica el formato entre editores):

.editorconfig

Copiar

```
root = true

[*]
charset = utf-8
end_of_line = lf
insert_final_newline = true
indent_style = space
indent_size = 4
trim_trailing_whitespace = true

[*.yaml,json,js,ts,vue]
indent_size = 2
```

Contenido base para `README.md`:

README.md

Copiar

```
# INF560 — Desarrollo Web Backend (Gestión 02/2026)

Repositorio personal de la asignatura.

- **Estudiante:** Nombre Apellido
- **Carrera:** Ingeniería Informática — UATF
- **Docente:** M. Sc. Huáscar Fedor Gonzales Guzmán

## Estructura

- `laboratorios/` — guías de laboratorio resueltas.
- `proyecto/` — proyecto acumulativo (eventos-app).
- `recursos/` — material de apoyo.
```

15 Ejercicio práctico integrador

Reúna todo lo anterior creando y publicando su repositorio del semestre. Siga los pasos en orden:

1. Cree una carpeta para el repositorio y entre en ella: `mkdir INF560-2026-02-Apellido` y luego `cd INF560-2026-02-Apellido`.
2. Inicialice el repositorio local: `git init`.
3. Cree los archivos base `README.md`, `.gitignore` y `.editorconfig` con el contenido de la Sección 14, y la estructura de carpetas.
4. Revise el estado del repositorio: `git status` (verá los archivos sin seguimiento).
5. Prepare todos los cambios: `git add ..`
6. Realice el primer commit: `git commit -m "Agrega estructura inicial del repositorio"`.
7. Vincule el repositorio remoto con la URL SSH: `git remote add origin git@github.com:usuario/INF560-2026-02-Apellido.git`.
8. Asegure que la rama principal se llame main: `git branch -M main`.
9. Publique los cambios: `git push -u origin main`.
10. Realice al menos dos commits adicionales (por ejemplo, agregando capturas en `laboratorios/gl01-entorno/evidencias/`) y publíquelos con `git push`.

Nota

Recargue la página de su repositorio en GitHub tras el push: debe ver el README renderizado y la estructura de carpetas. Ese enlace es su entregable.

16 Checklist de verificación final

Ejecute cada comando y confirme que obtiene una salida similar a la esperada:

Comando	Salida esperada (aprox.)
<code>git --version</code>	<code>git version 2.46.x</code>
<code>php -v</code>	<code>PHP 8.4.x (cli)</code>
<code>php -m</code>	<code>incluye pdo_pgsql, pgsql, mbstring</code>
<code>composer -V</code>	<code>Composer version 2.8.x</code>
<code>psql --version</code>	<code>psql (PostgreSQL) 18.x</code>
<code>node -v</code>	<code>v24.x.x</code>
<code>npm -v</code>	<code>11.x.x</code>
<code>code --version</code>	<code>1.x.x</code>
<code>ssh -T git@github.com</code>	<code>Hi usuario! You've successfully authenticated...</code>

17 Solución de problemas comunes

Síntoma	Causa probable y solución
'php' no se reconoce como comando	El PATH no incluye la carpeta de PHP, o no reabrió la terminal. Verifique la variable PATH y abra una nueva terminal.
could not find driver (PDO)	Las extensiones pdo_pgsql/pgsql no están habilitadas. Descoméntelas en php.ini (Windows) o instale php8.4-pgsql (Linux).
password authentication failed	Contraseña incorrecta del rol o método de autenticación distinto. Revise la contraseña y, en Linux, use <code>sudo -u postgres psql</code> .
Permission denied (publickey)	La llave SSH no se agregó al agente o a GitHub, o el remoto usa HTTPS en vez de SSH. Verifique con <code>ssh -T git@github.com</code> y la URL del remoto.
'nvm' no se reconoce (Windows)	Abra una nueva terminal como administrador tras instalar nvm-windows.
Composer se queda sin memoria	Ejecute con <code>COMPOSER_MEMORY_LIMIT=-1</code> antes del comando.

18 Entregable y criterios de evaluación

Entregable

- Enlace público al repositorio INF560-2026-02-Apellido en GitHub.
- En laboratorios/gl01-entorno/evidencias/: capturas de las verificaciones (php -v, psql --version, node -v, ssh -T git@github.com, etc.).

Rúbrica

Criterio	Puntaje
Todas las herramientas instaladas y verificadas (capturas de las versiones)	30
Repositorio con estructura de carpetas y archivos base (.gitignore, .editorconfig, README)	25
Commits con mensajes descriptivos y publicados (push)	20
Autenticación SSH funcional (captura de ssh -T)	15
Entrega en fecha y enlace correcto del repositorio	10
Total	100

19 Bibliografía y recursos

- Chacon, S. & Straub, B. (2014). Pro Git (2.^a ed.). Apress. <https://git-scm.com/book/es/v2>.
- Documentación oficial de PHP: <https://www.php.net/docs.php>.
- Documentación oficial de Composer: <https://getcomposer.org/doc/>.
- Documentación oficial de PostgreSQL 18: <https://www.postgresql.org/docs/18/>.
- Documentación oficial de Node.js: <https://nodejs.org/docs/latest-v24.x/api/>.
- Documentación de Laravel: <https://laravel.com/docs>.
- Guía de GitHub para SSH: <https://docs.github.com/es/authentication/connecting-to-github-with-ssh>.